

17.02.2026г.

Въведение в ООП

ООП
Лекция 1

Тема 1

Процедурен стил & ООП - какво подобрява ООП.

Основни принципи в ООП. Класове и обекти на високо ниво. Структури в езика C++. Namespaces. Enum classes.

1. Видове памет

- **Stack** - локалната памет, количеството заделена памет е определена по време на компиляция; заделя се в момента на дефиниция на променливите и се освобождава в момента на изход от scope-а, в която е дефинирана
- **Heap** - динамична памет (new/delete), заделя се и се освобождава по всяко време на изпълнение на програмата
- **Глобална (статична)** - тук се записват статичните/глобалните променливи
- **Program Code** - тук се пази нашия компилиран код

2. Процедурно програмиране - подходящ за малки програми, данните/функциите са отделени, логиката се разпределя на много функции.

проблеми: няма контрол върху достъпа до променливите, всеки може да промени данните

3. Какво е Обектно ориентирано програмиране?

Програмна парадигма - фундаментален стил на програмиране, дефинира се с редица от принципи, концепции и техники как да бъде имплементирана програмата

• OOP - програмна парадигма, при която системата се моделира като набор от обекти, които взаимодействат помежду си. Обектите са способни да получават съобщения, обработват данни и пращат съобщения на други обекти.

• При традиционното виждане програмата е списък от инструкции, които компютърът изпълнява.

• Основни идеи при OOP:
- данните + проверките са заедно
- има контрол кой какво да достъпва
- подходящо за големи системи

• Енкапсулация - защита на данните

4. Class - потребителски дефиниран тип данни, който обединява: данни (член-данни), функции (член-функции) и контрол на достъпа
- по подразбиране членовете са private

↓
основен
механизъм
при OOP

```
class ИмеНаКлас {  
    //членове  
};
```

- 1) Типова декларация
- 2) Област на обхват (scope)
- 3) Логическа единица за абстракция

5. Struct - същото като клас, но тук видимостта на ~~членовете~~ променливите е public
- членовете могат да бъдат от различен тип и с различна големина (при class също)

Примери за инициализиране:

```
Point p1 { 10, 20 };  
Point p2 = { 5, 7 };  
Point p3; // x и y не са инициализирани
```

```
Point p4 {  
    .x = 10,  
    .y = 20,  
};
```


свободни нумерации - тип, чиято стойност е ограничена до диапазон от стойности, който може да включва няколко изрично посочени константи (енумератори);

Стойностите на константите = стойности от интегрален тип (основен тип на енумерацията)

Разлика между енит и еним class
↳ по-добър, не позволява сравнение между различни типове

7. Namespaces - механизъм за организиране на декларации и наименования дефиниции в отделни логически области от имена пространства

Цел: избягване на конфликти на идентификатори, дефиниране на модулни граници

namespace {} - анонимни пространства от имена, всичко в тях е локално за текущия файл (не се вижда в други файлове, дори и с #include)

8. Създаване на динамични обекти

- Създават се в heap-а с new и се унищожават с delete.

Примери:

```
Person *p = new Person("Todor"); // създаване
p->greet(); // достъпване методите
delete p; // изтриване обекта
int n = 3;
Person *people = new Person[n] { {"Alice"}, {"Bob"}, {"Peter"} }; // създаване масив
for (int i = 0; i < n; i++)
    people[i].greet();
delete [] people; // освобождаване масив
```

2.

9. Влагане на обекти - клас съдържа обект от друг клас като
член-данна